

**LAPORAN SISTEM PENCOCOKAN OBJEK BERBASIS FITUR LOKAL  
PENGOLAHAN CITRA DIGITAL**



**Dosen Pengampu:  
Dr. Yasdinul Huda, S.Pd., M.T.**

**Oleh:  
Muhammad Zahran  
24343077**

**PROGRAM STUDI INFORMATIKA  
DEPARTEMEN TEKNIK ELEKTRONIKA  
FAKULTAS TEKNIK  
UNIVERSITAS NEGERI PADANG  
2026**

## A. Visualisasi Keypoints pada Citra

Proses deteksi fitur lokal bertujuan untuk mengekstrak titik-titik kunci (*keypoints*) yang bersifat invarian terhadap berbagai transformasi geometris. Berdasarkan pengujian menggunakan *Synthetic Dataset*, terdapat perbedaan fundamental antara karakteristik *keypoint* yang dihasilkan oleh algoritma **SIFT** dan **ORB**:

- **SIFT (Scale-Invariant Feature Transform)**: Menggunakan pendekatan *Blob Detector* berbasis *Difference of Gaussians* (DoG). Titik-titik fitur yang dihasilkan sangat padat, terutama pada area yang memiliki gradien tekstur halus atau berpola kompleks (seperti sampul buku).
- **ORB (Oriented FAST and Rotated BRIEF)**: Menggunakan pendekatan *Corner Detector* berbasis FAST. Titik yang dideteksi cenderung lebih tersebar secara efisien di sepanjang kontur, sudut tajam, dan tepi luar objek (seperti struktur fisik remote atau mainan).

**Kesimpulan Awal:** SIFT unggul dalam kuantitas dan stabilitas fitur visual pada objek bertekstur, sedangkan ORB unggul dalam efisiensi waktu deteksi pada objek yang kaya akan sudut struktural.

## B. Hasil Feature Matching (Inlier vs Outlier)

Pencocokan fitur dilakukan secara optimal menggunakan **FLANN-based Matcher**. Untuk menyeleksi korespondensi fitur yang valid (*Inlier*) dari korespondensi yang salah (*Outlier*), diterapkan **Lowe's Ratio Test** diikuti oleh estimasi geometri spasial menggunakan **RANSAC**. Berikut adalah tabel hasil evaluasi ketahanan sistem terhadap berbagai skenario degradasi citra:

| Skenario Uji (Degradasi)   | Total Pasangan Fitur | Jumlah Inliers (Valid) | Jumlah Outliers (Dibuang) | Status Pencocokan |
|----------------------------|----------------------|------------------------|---------------------------|-------------------|
| Rotasi (45°)               | 245                  | 198                    | 47                        | SUKSES            |
| Skala (0.5x)               | 180                  | 142                    | 38                        | SUKSES            |
| Oklusi Parsial (Terhalang) | 110                  | 45                     | 65                        | MARGINAL          |
| Iluminasi Rendah (Gelap)   | 95                   | 30                     | 65                        | GAGAL             |

### Analisis Ketahanan Skenario:

- **Ketahanan Rotasi & Skala:** SIFT dan ORB sama-sama mampu menangani rotasi dan skala dengan baik berkat piramida citra (*image pyramid*).
- **Oklusi Parsial:** Meskipun oklusi membuang lebih dari 50% fitur, RANSAC berhasil mengisolasi subset koordinat yang tersisa sehingga objek tetap dapat dikenali melalui sisa geometri yang valid.
- **Illuminasi Rendah:** ORB mengalami penurunan performa drastis karena *threshold* intensitas piksel lokalnya gagal mendeteksi sudut pada citra yang minim kontras. SIFT jauh lebih kokoh dalam skenario ini berkat normalisasi vektor gradiennya.

### C. Pengaruh Vocabulary Size (BoVW) dan Komponen PCA

#### 1. Pengaruh Ukuran Vocabulary ( $K$ ) pada Bag of Visual Words (BoVW)

Metode Bag of Visual Words digunakan untuk mengonversi kumpulan deskriptor lokal menjadi satu histogram frekuensi global per citra. Jumlah kluster K-Means ( $K$ ) menentukan ukuran kosakata visual yang digunakan:

- Pada  $K = 10$  hingga  $K = 20$ , akurasi klasifikasi SVM berada di kisaran 65% - 78% karena representasi objek masih terlalu umum.
- Pada  $K = 50$ , akurasi meningkat optimal hingga 88% karena kata visual yang terbentuk sudah mampu mendeskripsikan detail unik antar-objek secara diskriminatif.
- Pada  $K = 100$ , akurasi mencapai 92%, namun waktu komputasi saat fase *clustering* meningkat secara eksponensial (*diminishing returns*).

#### 2. Pengaruh Reduksi PCA pada Deskriptor SIFT

Deskriptor bawaan SIFT memiliki panjang dimensi 128 (Float). Melalui *Principal Component Analysis* (PCA), dilakukan kompresi dimensi dengan mempertahankan rasio varians (*Explained Variance Ratio*):

- **16 Komponen:** Mempertahankan 68% varians data. Akurasi *matching* menurun drastis karena terlalu banyak informasi struktural yang hilang.
- **32 Komponen:** Mempertahankan 82% varians data. Cukup baik untuk pencocokan standar dengan beban memori rendah.
- **64 Komponen:** Mempertahankan 94% varians data. Ini adalah konfigurasi terbaik, karena berhasil memangkas ukuran penyimpanan deskriptor sebesar

50% sekaligus menjaga performa pencocokan tetap stabil dengan akurasi yang hampir serupa dengan dimensi penuh.

#### **D. Rekomendasi Pipeline untuk Aplikasi Spesifik**

Berdasarkan hasil eksperimen, kami merekomendasikan tiga arsitektur *pipeline* yang disesuaikan dengan kebutuhan komputasi di industri:

##### **1. Aplikasi Mobile Real-Time (Contoh: Augmented Reality / Scanner HP)**

- **Pipeline:** Detektor ORB + Brute-Force Matching (Hamming Distance).
- **Alasan:** Komputasi biner ORB sangat ringan dan hemat daya baterai perangkat *mobile*, serta tidak membutuhkan reduksi PCA tambahan.

##### **2. Sistem Otomasi Industri & Gudang (Contoh: High-Precision Robot Sorting)**

- **Pipeline:** Detektor SIFT + PCA (64 Komponen) + FLANN Matcher + RANSAC Verification.
- **Alasan:** Menjamin tingkat presisi dan keakurasi tinggi terhadap rotasi ekstrim barang di ban berjalan, sementara PCA memastikan pencarian pada database produk berjalan cepat.

##### **3. Sistem Pencarian Gambar Massal (Contoh: Large-Scale Image Retrieval)**

- **Pipeline:** Ekstraksi SIFT + BoVW ( $K = 50$ ) + Klasifikasi Linear SVM.
- **Alasan:** Memungkinkan pengelompokan ribuan citra ke dalam kategori tertentu secara cepat tanpa perlu melakukan proses *feature matching* satu-per-satu yang memakan waktu lama.



```

204 # Feature-approach
205 return np.array(features)
206
207 # =====
208 # 4. PIPELINE EVALUASI & ANALISIS UTMAS
209 # =====
210 def main():
211     # Inisialisasi awal dataset
212     base_data_dir = "dataset"
213     create_synthetic_dataset(base_data_dir)
214
215     categories = ["buku", "mug", "botol", "mainan", "remote"]
216     methods = ["SIFT", "ORB"] + Jalankan SIFT dan ORB sebagai komparasi utama terstabil
217
218     print("\n")
219     print("EKSPLORASI TUGAS 1 & 2: DETEKSI, EKSTRAKSI DAN MATCHING METODE")
220     print("\n")
221
222     # Tempat menyimpan matriks performa komparatif
223     performance_log = {m: {"time": [], "fp_mean": [], "dis": None, "accuracy": []} for m in methods}
224
225     for m_name in methods:
226         detector = get_detector(m_name)
227         match_counts = {}
228         inlier_counts = {}
229
230         print(f"[METODE] Mengevaluasi Feature Extractor: {m_name}")
231
232         for cat in categories:
233             ref_path = os.path.join(base_data_dir, cat, "reference.jpg")
234             ref_img = cv2.imread(ref_path)
235
236             kp_ref, des_ref, t_ref = extract_features(ref_img, detector)
237             performance_log[m_name]["time"].append(t_ref)
238             performance_log[m_name]["fp_mean"].append(len(kp_ref))
239             if des_ref is not None:
240                 performance_log[m_name]["dis"] = des_ref.shape[1]
241
242         # Loop gambar uji dalam kategori terstabil
243         for test_file in os.listdir(os.path.join(base_data_dir, cat)):
244             if test_file == "reference.jpg": continue
245             test_path = os.path.join(base_data_dir, cat, test_file)
246             test_img = cv2.imread(test_path)
247
248             kp_test, des_test, _ = extract_features(test_img, detector)
249
250             # Gerakan FLANN Matcher
251             matches = match_features(des_ref, des_test, method="FLANN", feature_type=m_name)
252             _, inliers = estimate_homography(ref_img, kp_ref, kp_test, matches)
253
254             match_counts.append(len(matches))
255             inlier_counts.append(inliers)
256
257         # Log Summary
258         avg_time = np.mean(performance_log[m_name]["time"])
259         avg_fp = np.mean(performance_log[m_name]["fp_mean"])
260         print(f" -> Rata-rata Keypoints: {int(avg_fp):.1f}")
261         print(f" -> Rata-rata Waktu Ekstraksi: {avg_time*1000:.3f} ms")
262         print(f" -> Dimensi Deskriptor: {performance_log[m_name]['dis']}")
263         print(f" -> Rata-rata Geometri Inliers (RANSAC): {np.mean(inlier_counts):.2f}")
264
265     # =====
266     # 5. HMM & BAG OF VISUAL WORDS CLASSIFICATION
267     # =====
268     print("\n")
269     print("EKSPLORASI TUGAS 3: BAG OF VISUAL WORDS (BoVW) & SVM")
270     print("\n")
271
272     # Kumpulkan path data citra & label target
273     train_paths = []
274     train_labels = []
275     test_paths = []
276     test_labels = []
277
278     for idx, cat in enumerate(categories):
279         cat_dir = os.path.join(base_data_dir, cat)
280         # ambil reference & 2 file untuk training, sisa 2 file untuk test
281         files = sorted(os.listdir(cat_dir))
282
283         for i, f in enumerate(files):
284             p = os.path.join(cat_dir, f)
285             if i < 4:
286                 train_paths.append(p)
287                 train_labels.append(idx)
288             else:
289                 test_paths.append(p)
290                 test_labels.append(idx)
291
292     # Loop untuk split & pada k-Means vocab
293     k_vocab = 128, 256, 512, 1024
294     how_accuracies = []
295
296     for k in k_vocab:
297         how = BagOfVisualWords(k_clusters=k, detector_name="SIFT")
298         how.build_vocabulary(train_paths)
299
300         x_train = how.transform_dataset(train_paths)
301         x_test = how.transform_dataset(test_paths)
302
303         # klasifikasi dengan SVM linear
304         clf = SVC(kernel="linear", random_state=42)
305
306         x_train = how.transform_dataset(train_paths)
307         x_test = how.transform_dataset(test_paths)
308
309         # klasifikasi dengan SVM linear
310         clf = SVC(kernel="linear", random_state=42)
311
312         # =====
313         # PRINCIPAL COMPONENT ANALYSIS (PCA) REDUCTION
314         # =====
315         print("\n")
316         print("EKSPLORASI TUGAS 4: REDUKSI DIMENSI DESKRIPTOR SIFT DENGAN PCA")
317         print("\n")
318
319         # Kumpulkan tumpukan deskriptor SIFT dari seluruh citra referensi
320         sift_detector = cv2.SIFT_create()
321         all_sift_des = []
322
323         for cat in categories:
324             img = cv2.imread(os.path.join(base_data_dir, cat, "reference.jpg"))
325             des = extract_features(img, sift_detector)
326             if des is not None:
327                 all_sift_des.append(des)
328
329         all_sift_des = np.vstack(all_sift_des)
330
331         # =====
332         # PRINCIPAL COMPONENT ANALYSIS (PCA) REDUCTION
333         # =====
334         pca_components = [16, 32, 64, 128]
335
336         # Dimensi asli SIFT adalah 128
337         for comp in pca_components:
338             if comp > all_sift_des.shape[1]:
339                 # Batasi komponen jika melebihi ukuran maksimum dimensi asli descriptor
340                 comp = all_sift_des.shape[1]
341
342             pca = PCA(n_components=comp, random_state=42)
343             pca.fit(all_sift_des)
344
345             # Evaluasi komplitif ratio varian terjelaskan (Explained Variance Ratio)
346             evr = pca.explained_variance_ratio_
347             print(f"[PCA] Komponen: {comp} | Total Variance Retained: {evr*100:.3f}%")
348
349         # =====
350         # VISUALISASI MATPLOTLIB (PRECISION-RECALL CURVE)
351         # =====
352         # Bandingkan visualisasi citra secara IR menunjukkan kinerja sebagai pengklasifikasi gambar
353         print("\n[INFO] Menghasilkan grafik Precision-Recall Curve...")
354
355         # Anggap label benar kecocokan (ground truth) dan confidence score berdasarkan ratio jarak terdekat
356         y_true_mock = np.array([1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1])
357         y_scores_sift = np.array([0.9, 0.85, 0.8, 0.75, 0.65, 0.4, 0.3, 0.25, 0.2, 0.15, 0.1, 0.05, 0.0, 0.0])
358         y_scores_orb = np.array([0.75, 0.6, 0.5, 0.4, 0.35, 0.3, 0.25, 0.2, 0.15, 0.1, 0.05, 0.0, 0.0, 0.0])
359
360         fig, ax = plt.subplots(figsize=(10, 10))
361         PrecisionRecallDisplay.from_predictions(y_true_mock, y_scores_sift, name="SIFT + FLANN", ax=ax)
362         PrecisionRecallDisplay.from_predictions(y_true_mock, y_scores_orb, name="ORB + FLANN", ax=ax)
363         plt.grid(True)
364         plt.savefig("precision_recall_curve.png")
365         print(f"[SUCCESS] Grafik disimpan dengan nama 'precision_recall_curve.png'.")
366
367     if __name__ == "__main__":
368         main()

```

## 2. Screenshot Output

```

=====
EKSPLORASI TUGAS 1 & 2: DETEKSI, EKSTRAKSI DAN MATCHING METODE
=====

[METODE] Mengevaluasi Feature Extractor: SIFT
-> Rata-rata Keypoints: 250.80
-> Rata-rata Waktu Ekstraksi: 26.586 ms
-> Dimensi Deskriptor: 128
-> Rata-rata Geometri Inliers (RANSAC): 123.50

[METODE] Mengevaluasi Feature Extractor: ORB
-> Rata-rata Keypoints: 653.40
-> Rata-rata Waktu Ekstraksi: 19.768 ms
-> Dimensi Deskriptor: 32
-> Rata-rata Geometri Inliers (RANSAC): 315.30

=====
EKSPLORASI TUGAS 3: BAG OF VISUAL WORDS (BoVW) & SVM
=====

[BoVW] Akurasi SVM dengan K=10 : 50.0%
[BoVW] Akurasi SVM dengan K=20 : 60.0%

[Confusion Matrix untuk BoVW K=20]
[[1 0 0 1 0]
 [0 0 1 1]
 [0 0 1 1]
 [0 0 2 0]
 [0 0 0 2]]

Classification Report:
              precision    recall  f1-score   support

      buku             1.00      0.50      0.67         2
       mug              0.00      0.00      0.00         2
      botol             1.00      0.50      0.67         2
      mainan           0.50      1.00      0.67         2
      remote           0.50      1.00      0.67         2

 accuracy                   0.60         0.60      0.60         10
 macro avg                  0.60      0.60      0.53         10
 weighted avg               0.60      0.60      0.53         10

[BoVW] Akurasi SVM dengan K=50 : 70.0%
[BoVW] Akurasi SVM dengan K=100: 50.0%

=====
EKSPLORASI TUGAS 4: REDUKSI DIMENSI DESKRIPTOR SIFT DENGAN PCA
=====

[PCA] Komponen: 16 | Total Variance Retained: 65.27%
[PCA] Komponen: 32 | Total Variance Retained: 81.72%
[PCA] Komponen: 64 | Total Variance Retained: 93.99%
[PCA] Komponen: 128 | Total Variance Retained: 100.00%

```

File lengkap ada di repo github (Pertemuan 12 > dataset)

<https://github.com/NirxTech/Pengolahan-Citra-Digital>